

UNSW, School of Mathematics and Statistics

Profs Josef Dick and Kassem Mustapha

MATH3311/MATH5335

Topic 05 – Cubic Splines and Sparse Matrices

- 1 Polynomials
 - Horner's method
 - Weierstrass approximation theorem
 - Polynomial interpolation
- 2 Data fitting – Non-parametric methods
 - Cubic splines

- 3 Sparse matrices
 - Definition of sparse matrix
 - Example of hidden sparsity
 - Boundary value problem
 - Banded matrices: LU and Cholesky
 - Fill-in

Polynomials - Horner's method

A polynomial of **degree** n has $n + 1$ parameters $a_0, a_1, \dots, a_n$:

$$p(t) = \sum_{k=0}^n a_k t^k = a_0 + a_1 t + a_2 t^2 + \dots + a_n t^n.$$

- Polynomials are ideal for computer evaluation: only multiplication and addition are needed
- **Flops**: n multiplications and n additions, so $2n$ flops.
- The **monomial** basis $1, t, t^2, t^3, \dots, t^n$ is **not** ideal.
- An equivalent expression is **Horner's method** (it reduces rounding errors):

$$p(t) = a_0 + t(a_1 + t(a_2 + t(a_3 + \dots + t(a_{n-1} + ta_n)) \dots))$$

- MATLAB function **horner.m**, Python function **horner.py**

Polynomial approximations

Theorem (Weierstrass approximation theorem, 1885)

Let f be a continuous function on $[a, b]$. For each $\epsilon > 0$, there exists a polynomial p such that

$$|f(t) - p(t)| < \epsilon \quad \text{for all } t \in [a, b].$$

*This assumes that the values of $f(t)$ can be determined **exactly** for all $t \in [a, b]$.*

- Any continuous function on $[a, b]$ can be approximated arbitrarily well by a polynomial.
- Polynomials are easy to evaluate, differentiate, and integrate.

Polynomial interpolation

- **Polynomial interpolation:** the polynomial $\sum_{j=0}^n x_j t^j$ interpolates the data (t_i, y_i) for $i = 0, \dots, n$, if

$$\sum_{j=0}^n x_j t_i^j = x_0 + x_1 t_i + x_2 t_i^2 + \dots + x_n t_i^n = y_i \quad \text{for } i = 0, \dots, n.$$

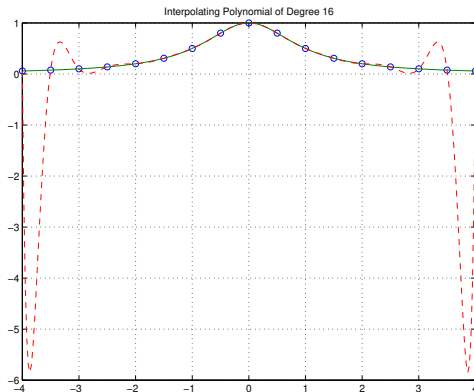
This leads to the linear system: (A is the Vandermonde matrix)

$$\underbrace{\begin{bmatrix} 1 & t_0 & t_0^2 & \dots & t_0^n \\ \vdots & \vdots & \vdots & \dots & \vdots \\ 1 & t_n & t_n^2 & \dots & t_n^n \end{bmatrix}}_A \underbrace{\begin{bmatrix} x_0 \\ \vdots \\ x_n \end{bmatrix}}_x = \underbrace{\begin{bmatrix} y_0 \\ \vdots \\ y_n \end{bmatrix}}_y$$

- Unless n is small, the polynomial interpolant should not be used because of numerical instability and spurious oscillations near the end points.

Example

Example. Polynomial approximation of the Runge function $f(t) = \frac{1}{1+t^2}$ by interpolating at equally spaced points.



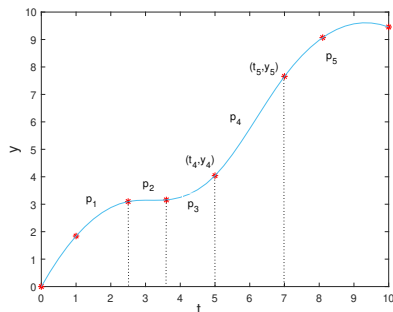
See MATLAB `runge.m` or Python `runge.py`

Piecewise polynomial interpolation

Instead of a single global polynomial interpolant of high degree n , we use piecewise polynomials, each of low degree.

Example

Piecewise polynomials of degree 3 are used to interpolate the given data (*) instead of a single polynomial of degree 7.



Cubic spline

Cubic splines are used in a wide range of applications. In finance, they provide a smooth approximation to the missing data (data smoothing).

Definition

Let $a = t_0 < t_1 < t_2 < \dots < t_n = b$. A function $s : [a, b] \rightarrow \mathbb{R}$ is a **cubic spline** if there exist polynomials $s_1, s_2, \dots, s_n$, each of degree at most 3 (order 4), such that

- ① $s(t) = s_i(t)$ for $t_{i-1} \leq t \leq t_i$ for $i = 1, 2, \dots, n$;
- ② $s \in \mathcal{C}^2[a, b]$, that is, s, s' and s'' are continuous on $[a, b]$.

The points $t_0, t_1, \dots, t_n$ are called **knots** or **breakpoints**.

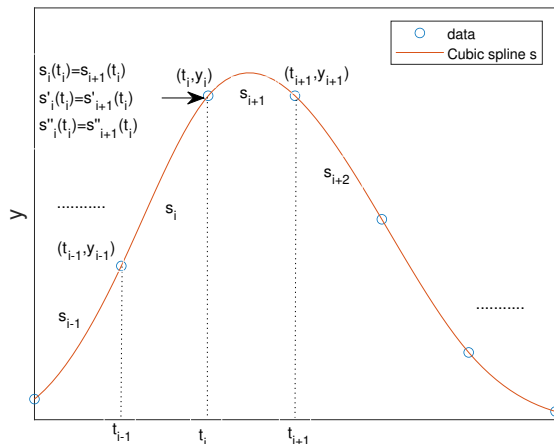
- For $i = 1, 2, \dots, n$,

$$s_i(t) = a_i + b_i t + c_i t^2 + d_i t^3 \quad \text{for } t_{i-1} \leq t \leq t_i.$$

- To find $s(t)$, it suffices to find a_i, b_i, c_i, d_i for $i = 1, 2, \dots, n$; thus there are $4n$ coefficients in total.

Cubic spline interpolation- graphical illustrations

Cubic spline interpolation offers a higher degree of smoothness and accuracy. Below is a graphical illustration of the cubic spline s interpolating the data points.



Cubic spline interpolation

- **Interpolation** gives $n + 1$ conditions:

$$s(t_i) = y_i, \quad i = 0, 1, 2, \dots, n.$$

- **Continuity at the interior knots** gives $3(n - 1)$ conditions: for $i = 1, 2, \dots, n - 1$,

$$s_i(t_i) = s_{i+1}(t_i), \quad s'_i(t_i) = s'_{i+1}(t_i), \quad s''_i(t_i) = s''_{i+1}(t_i).$$

- Since $(n + 1) + 3(n - 1) = 4n - 2$, two more equations are needed to determine s uniquely.
- We impose boundary conditions at the endpoints a and b . Standard choices are:
 - **Natural spline:** $s''(a) = 0 = s''(b)$.
 - **Clamped spline:** $s'(a) = y'_a$ and $s'(b) = y'_b$.
 - **Not-a-knot spline:** $s'''_1(t_1) = s'''_2(t_1)$ and $s'''_{n-1}(t_{n-1}) = s'''_n(t_{n-1})$.
As s is piecewise cubic, these conditions imply
 - $s_1 \equiv s_2$ on $[t_0, t_2]$ and $s_{n-1} \equiv s_n$ on $[t_{n-2}, t_n]$.

Cubic spline interpolation - parametrization

- In practice, and for simplicity, if $h = h_i = t_i - t_{i-1}$ for $i = 1, \dots, n$, we write the polynomial s_i in the form: for $t \in [t_{i-1}, t_i]$,

$$s_i(t) = \frac{M_{i-1}}{6h}(t_i - t)^3 + \frac{M_i}{6h}(t - t_{i-1})^3 \\ + \left(\frac{y_{i-1}}{h} - \frac{M_{i-1}h}{6} \right)(t_i - t) + \left(\frac{y_i}{h} - \frac{M_ih}{6} \right)(t - t_{i-1}),$$

where $M_0, M_1, \dots, M_n$ are to be determined.

- It is clear that the interpolation condition $s(t_i) = y_i$ for $i = 0, \dots, n$ holds.
- To satisfy the conditions $s'_i(t_i) = s'_{i+1}(t_i)$ for $i = 1, \dots, n-1$, we differentiate and let $d_i = (y_i - y_{i-1})/h$,

$$s'_i(t) = -\frac{M_{i-1}}{2h}(t_i - t)^2 + \frac{M_i}{2h}(t - t_{i-1})^2 + d_i + (M_{i-1} - M_i)\frac{h}{6},$$

$$s'_{i+1}(t) = -\frac{M_i}{2h}(t_{i+1} - t)^2 + \frac{M_{i+1}}{2h}(t - t_i)^2 + d_{i+1} + (M_i - M_{i+1})\frac{h}{6}.$$

$$s'_i(t_i) = s'_{i+1}(t_i) \implies \frac{M_i}{2}h + d_i + (M_{i-1} - M_i)\frac{h}{6} = -\frac{M_i}{2}h + d_{i+1} + (M_i - M_{i+1})\frac{h}{6}.$$

Cubic spline interpolation - parametrization (continued)

Thus, for $i = 1, 2, \dots, n-1$,

$$\frac{h}{6} M_{i-1} + \frac{2h}{3} M_i + \frac{h}{6} M_{i+1} = d_{i+1} - d_i. \quad (1)$$

We have $n-1$ linear equations in the $n+1$ unknowns $M_0, M_1, \dots, M_n$.

Note:

- The second derivatives

$$s_i''(t) = \frac{M_{i-1}}{h}(t_i - t) + \frac{M_i}{h}(t - t_{i-1}),$$
$$s_{i+1}''(t) = \frac{M_i}{h}(t_{i+1} - t) + \frac{M_{i+1}}{h}(t - t_i),$$

and hence, $s_i''(t_i) = s_{i+1}''(t_i) = M_i$ for $i = 1, \dots, n-1$. Thus, the second-derivative continuity conditions are satisfied.

- For unique values of $M_0, M_1, \dots, M_n$, we need two more (independent) linear equations in addition to the set of $n-1$ linear equations in (1).

Cubic spline interpolation - parametrization (continued)

Example

- Let $n = 5$, and consider the case of natural cubic spline interpolant. Thus, $s_1''(t_0) = 0 = s_5'(t_5)$, which implies $M_0 = 0 = M_5$.
- $\frac{h}{6} M_{i-1} + \frac{2h}{3} M_i + \frac{h}{6} M_{i+1} = d_{i+1} - d_i$, $i = 1, 2, 3, 4$, is equivalent to

$$\frac{h}{6} \begin{bmatrix} 4 & 1 & & \\ 1 & 4 & 1 & \\ & 1 & 4 & 1 \\ & & 1 & 4 \end{bmatrix} \begin{bmatrix} M_1 \\ M_2 \\ M_3 \\ M_4 \end{bmatrix} = \begin{bmatrix} d_2 - d_1 \\ d_3 - d_2 \\ d_4 - d_3 \\ d_5 - d_4 \end{bmatrix}$$

The coefficient matrix is

- symmetric, tridiagonal, diagonally dominant, and positive-definite

Tridiagonal linear system: the Thomas algorithm can factor and solve it in $O(n)$ operations. See MATLAB `tridiag.m` or Python `tridiag.py`

Cubic spline interpolation - Example

Example (Approximate Runge function)

Use cubic spline interpolation (at equally spaced points with $h = 0.5$) to approximate $f(t) = \frac{1}{1+t^2}$ over $[-4, 4]$

MATLAB `runge.m`

```
%spline use the
%not-a-knot condition.

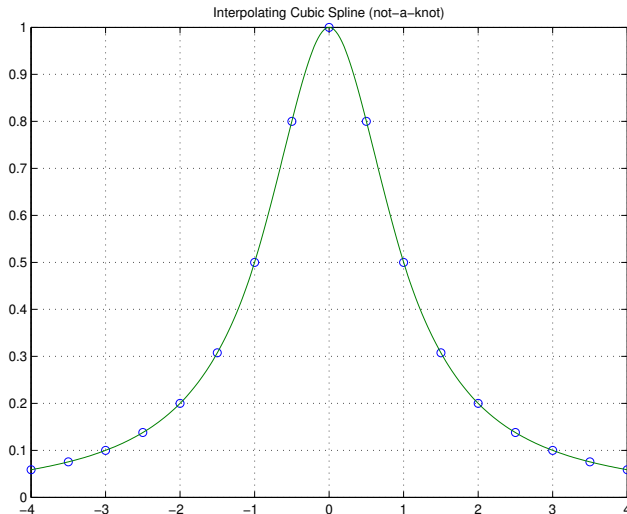
t=linspace(-4,4,17);
f=@(t) 1./(1 +t.^2);
y=f(t);
tt=linspace(-4,4,401);
yy=spline(t,y,tt);
plot(t, y, 'o', tt,yy)
```

Python `runge.py`

```
#CubicSpline use the not-a-knot condition.

from numpy import linspace
from scipy.interpolate import CubicSpline
from matplotlib.pyplot import plot
t = linspace(-4, 4, 17)
f = lambda t: 1 / ( 1 + t**2 )
y=f(t)
tt = linspace(-4, 4, 401)
yy = CubicSpline(t, y)(tt)
plot(t, y, 'o', tt, yy)
```

Cubic spline interpolation - Example (continued)



See MATLAB [runge.m](#) or Python [runge.py](#)

Sparse matrices

Many large matrices that arise in applications have a high proportion of their entries equal to zero.

- The matrix A is **sparse** if the **zero elements** of A can be exploited to improve efficiency (storage, time) [Wilkinson]
- The **sparsity** of $A \in \mathbb{R}^{m \times n}$ is the number of zero elements in A divided by the total number of elements (mn).
- The **density** of $A \in \mathbb{R}^{m \times n}$ is the number of nonzero elements in A divided by mn .

Example (Density)

Compute the density of $A \in \mathbb{R}^{n \times n}$ if A is diagonal, tridiagonal, or triangular

Solution

- *Diagonal matrix: density $\approx n/n^2 \approx 1/n$*
- *Tridiagonal matrix: density $\approx (3n - 2)/n^2 \approx 3/n$*
- *Triangular matrix: density $\approx [n(n + 1)/2]/n^2 = 1/2 + 1/(2n) \approx 50\%$*

Hidden sparsity

- Sometimes, a matrix can possess “hidden sparsity”.

Example

let B be an $n \times r$ matrix, where $r \ll n$, and let the $n \times n$ matrix

$$A = I + BB^T.$$

This matrix is generally dense. If $\mathbf{x} \in \mathbb{R}^n$, then to compute $A\mathbf{x}$ we do not need to form A explicitly; instead, we can compute

$$\mathbf{x} + B(B^T \mathbf{x}).$$

Boundary value problem (BVP)

Consider the two-point BVP (Note: $u'' = \frac{d^2 u}{dx^2}$):

$$-u'' = f(x) \quad \text{for } 0 < x < 1, \quad \text{with } u(0) = \alpha \text{ and } u(1) = \beta.$$

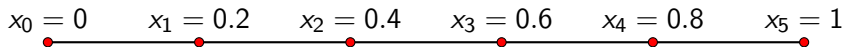
Here, the function f and the constants α and β are given data, and we want to solve (numerically) for u over the domain $[0, 1]$.

Solution

For the numerical solution, set up a (uniform) grid:

$$x_j = j\Delta x \quad \text{for } j = 0, 1, \dots, n+1, \quad \text{with } \Delta x = \frac{1}{n+1}.$$

For instance, if $n = 4$, ($\Delta x = 0.2$), we have the following 6 grid points:



BVP (continued)

Using a Taylor series expansion (with remainder $O((\Delta x)^4)$),

$$u(x_{j+1}) = u(x_j) + \Delta x u'(x_j) + \frac{(\Delta x)^2}{2} u''(x_j) + \frac{(\Delta x)^3}{6} u'''(x_j) + O((\Delta x)^4),$$

$$u(x_{j-1}) = u(x_j) - \Delta x u'(x_j) + \frac{(\Delta x)^2}{2} u''(x_j) - \frac{(\Delta x)^3}{6} u'''(x_j) + O((\Delta x)^4).$$

Adding these gives

$$u''(x_j) = \frac{u(x_{j+1}) - 2u(x_j) + u(x_{j-1}))}{(\Delta x)^2} + O((\Delta x)^2),$$

and thus, for a small Δx ,

$$-\frac{u(x_{j+1}) - 2u(x_j) + u(x_{j-1}))}{(\Delta x)^2} \approx f(x_j).$$

Motivated by this, we look for $U_j \approx u(x_j)$ by requiring that

$$-\frac{U_{j+1} - 2U_j + U_{j-1}}{(\Delta x)^2} = f(x_j) \quad \text{for } 1 \leq j \leq n, \quad \text{with } U_0 = \alpha \text{ and } U_{n+1} = \beta.$$

Eliminating U_0 and U_{n+1} leaves n equations for n unknowns $U_1, U_2, \dots, U_n$.

BVP (continued) MATLAB `bvp1d.m`, Python `bvp1d.py`

To proceed, we have to solve the following $n \times n$ linear system:

$$\begin{aligned}
 2U_1 - U_2 &= f(x_1)(\Delta x)^2 + \alpha, \\
 -U_1 + 2U_2 - U_3 &= f(x_2)(\Delta x)^2, \\
 &\vdots \\
 -U_{n-2} + 2U_{n-1} - U_n &= f(x_{n-1})(\Delta x)^2, \\
 -U_{n-1} + 2U_n &= f(x_n)(\Delta x)^2 + \beta.
 \end{aligned}
 \implies$$

$$\underbrace{\begin{bmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 & -1 \\ & & & -1 & 2 \end{bmatrix}}_{A \in \mathbb{R}^{n \times n}} \underbrace{\begin{bmatrix} U_1 \\ U_2 \\ \vdots \\ U_n \end{bmatrix}}_{\mathbf{U}} = \underbrace{\begin{bmatrix} f(x_1)(\Delta x)^2 + \alpha \\ f(x_2)(\Delta x)^2 \\ \vdots \\ f(x_{n-1})(\Delta x)^2 \\ f(x_n)(\Delta x)^2 + \beta \end{bmatrix}}_{\mathbf{b}}$$

A is tridiagonal, Toeplitz, symmetric, and positive-definite.

BVP (continued)

For large n , storing A as an $n \times n$ array is very inefficient, since the proportion of nonzero entries ($(3n - 2)/n^2 \approx 3/n$) is small. For instance, if $n = 100$ then about 97% of the storage is wasted. MATLAB and Python provide sparse array type. You can build A and solve $A\mathbf{U} = \mathbf{b}$ as:

```
% MATLAB
f = @(x) .....
Dx = 1/(n+1);
x = Dx*[1:n];
b = Dx^2*f(x)';
b(1) = b(1) + alpha; b(n) = b(n) + beta;
e = ones(n,1);
A = spdiags([-e 2*e -e],[-1 0 1], n, n); % Sparse array
U= A\b % Sparse solve is more efficient
```

BVP (continued)

```
# Python
```

```
from numpy import linspace, ones
```

```
from scipy.sparse import spdiags
```

```
from scipy.sparse.linalg import spsolve
```

```
f = lambda x: .....
```

```
Dx = 1/(n+1)
```

```
x = linspace(0, 1, n+2)
```

```
b = Dx**2 * f(x[1:n+1])
```

```
b[0] += alpha
```

```
b[n-1] += beta
```

```
e = ones(n);
```

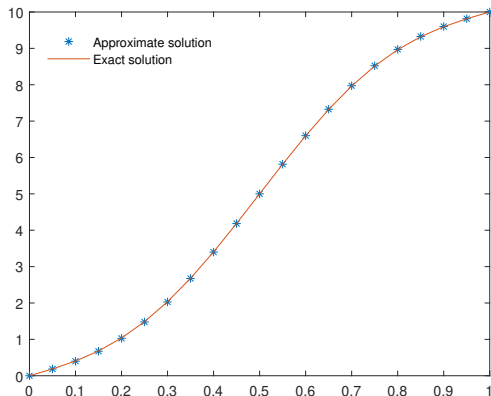
```
A = spdiags([-e, 2*e, -e], [-1, 0, 1], n, n) # sparse array
```

```
U = spsolve(A, b) # Sparse solve is more efficient
```

BVP (continued)

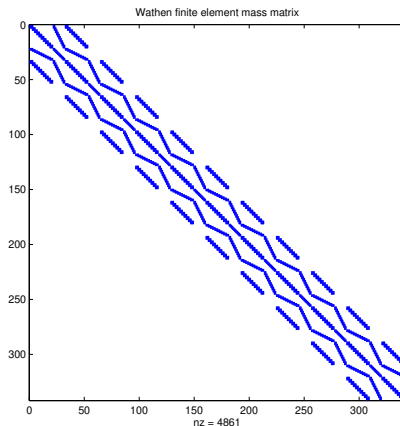
Example (Comparison between exact and approximate solutions)

Choose $\alpha = 0$, $\beta = 10$, and $f(x) = -40 \sin(2\pi x)$. Then,
 $u(x) = 10x - (10/\pi^2) \sin(2\pi x)$. For the approximate solution, fix $n = 19$.



Banded matrices

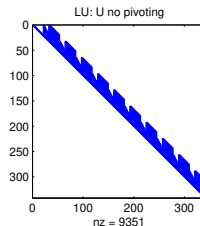
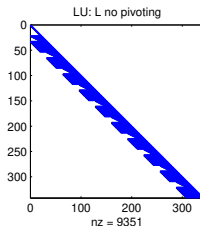
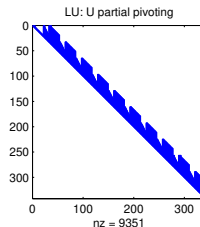
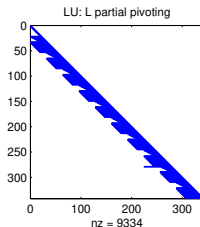
Banded matrices: nonzero (nz) elements close to the main diagonal. For example, the Wathen finite-element mass matrix.



See MATLAB `wathen.m` or Python `wathen.py`

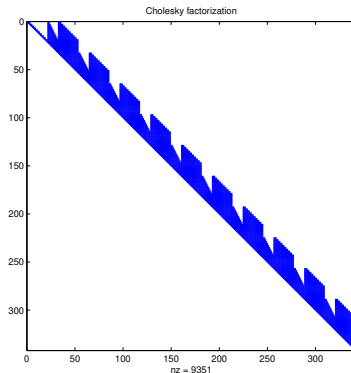
Banded matrices (continued)

Consider an LU-factorization $A = LU$. Then, L has the same lower bandwidth as A and U has the same upper bandwidth as A . However, pivoting can upset the band structure.



Banded matrices (continued)

If A is symmetric and positive-definite, then we can consider the Cholesky factorization $A = R^T R$. The matrix R has the same upper bandwidth as A .



In both LU and Cholesky factorizations, the triangular factor is often less sparse than the original matrix; this effect is known as **fill-in**.

Fill-in

Fill-in is the creation of new nonzero elements during matrix factorization or elimination.

- **Some reorderings** of rows/equations and/or columns/variables can reduce **fill-in**. This may conflict with partial pivoting, which can help **numerical stability**.
- For a **sparse** matrix A , we can try to reduce fill-in in the triangular factors of A by reordering the rows and columns of A so that $PAQ = LU$, where P and Q are row and column permutation matrices, respectively.
- MATLAB ($[L, U, P, Q] = \text{lu}(A)$) provides functions that compute P and Q so that the triangular factors L and U have reduced fill-in. See **splu** in Python.
- $PAQ = LU$ is often more time- and memory-efficient than $PA = LU$ because the additional column permutation Q can reduce fill-in in the sparse factors L and U .

Fill-in (continued)

Example (west0479 data matrix from Chemical engineering plant models)

$A = \text{west0479}$ (479-by-479 matrix) $PA = L_1 U_1$ and $PAQ = L_2 U_2$.

